

Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture

Project: Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture

Services: VPC, EC2, RDS, S3, IAM, Terraform

Description: This project demonstrates the implementation of a complete AWS 3-tier architecture using Terraform as an Infrastructure as Code (IaC) tool. The infrastructure includes networking components such as VPC, subnets, route tables, and internet gateway along with compute, database, storage, and security services including EC2, RDS, S3, and IAM. Terraform automates the provisioning and management of AWS resources, enabling reusable, scalable, and efficient infrastructure deployment.

Skills: Infrastructure as Code (IaC), Terraform, AWS Networking, Cloud Security, Automation, EC2 Deployment, RDS Configuration, IAM Roles, S3 Integration, Cloud Architecture.

Definition: The Infrastructure as Code (IaC) AWS 3-Tier Architecture project is designed to automate the deployment of a secure and scalable cloud infrastructure using Terraform. The architecture follows a 3-tier design consisting of presentation, application, and database layers. AWS services such as VPC, EC2, RDS, S3, and IAM are provisioned using Terraform scripts to ensure consistency, automation, and simplified infrastructure management.

Scope of the Project: The scope of this project is to design and implement a complete AWS 3-tier architecture using Terraform as the Infrastructure as Code tool. The project automates the provisioning of AWS networking, compute, storage, and database resources.

This project covers:

- Custom VPC Creation: Configures a dedicated Virtual Private Cloud with public and private subnets.
- Public and Private Subnets: Separates internet-facing resources from internal database resources for improved security.
- EC2 Web Server Deployment: Launches an Ubuntu EC2 instance configured with Apache web server hosting a custom HTML application.
- Amazon RDS MySQL Database: Creates a private MySQL database instance for backend data storage.
- Amazon S3 Integration: Provides scalable object storage for project-related storage requirements.
- IAM Role and Security Configuration: Implements IAM roles and security groups for secure communication between services.
- Terraform Automation: Uses Terraform scripts to automate infrastructure deployment and management.

Methodology:

1. Terraform + AWS – Best Method

- Uses Infrastructure as Code (IaC) to provision AWS resources automatically.
- Automates creation of VPC, EC2, RDS, S3, IAM, route tables, and security groups.
- Provides reusable and scalable infrastructure deployment.
- Reduces manual configuration errors and improves consistency.
- Supports version control and infrastructure management.
- Industry-standard DevOps approach suitable for production environments.
- Free-tier friendly when resources are configured carefully.

2. Manual AWS Console Deployment – Alternative Method

- Resources can also be created manually using AWS Management Console.
- Provides learning experience for understanding AWS services individually.
- Requires manual configuration of networking, security, and compute resources.
- More time-consuming and prone to configuration inconsistencies.
- Difficult to maintain and scale compared to Infrastructure as Code solutions.
- Best suited for beginners learning AWS fundamentals before automation.

AWS Services used in the Project:

1. Amazon VPC (Virtual Private Cloud)

- **Definition:** Amazon VPC is a service that enables users to create isolated virtual networks within AWS cloud infrastructure.
- **Purpose:** Provides secure and isolated networking environment for AWS resources.
- **Role:** Hosts public and private subnets, route tables, and internet gateway for the 3-tier architecture.

2. Amazon EC2 (Elastic Compute Cloud)

- **Definition:** Amazon EC2 is a cloud computing service that provides scalable virtual servers.
- **Purpose:** Hosts the web application and Apache web server.
- **Role:** Serves as the application layer in the 3-tier architecture and communicates with RDS database securely.

3. Amazon RDS (Relational Database Service)

- **Definition:** Amazon RDS is a managed relational database service that simplifies database setup and maintenance.
- **Purpose:** Provides managed MySQL database service.
- **Role:** Stores backend application data securely inside the private subnet.

4. Amazon S3 (Simple Storage Service)

- **Definition:** Amazon S3 is a scalable object storage service used for storing and retrieving data.
- **Purpose:** Stores project-related files and data securely.
- **Role:** Integrated with EC2 through IAM role permissions.

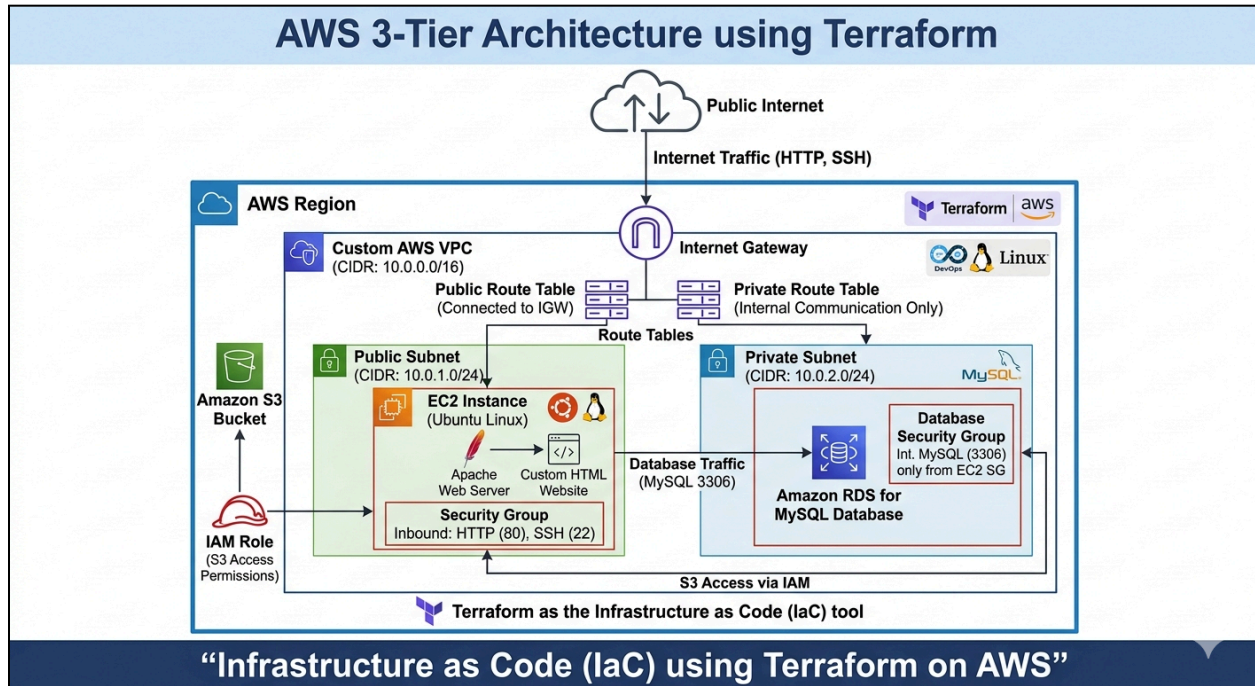
5. AWS IAM (Identity and Access Management)

- **Definition:** AWS IAM enables secure access management for AWS resources and services.
- **Purpose:** Provides secure authentication and authorization.
- **Role:** IAM role attached to EC2 instance allows secure access to S3 without storing access keys.

6. Terraform

- **Definition:** Terraform is an open-source Infrastructure as Code tool developed by HashiCorp.
- **Purpose:** Automates provisioning and management of AWS infrastructure using configuration files.
- **Role:** Deploys and manages the entire AWS 3-tier architecture automatically.

Architecture Diagram:



Implementation Steps to Do the Project:

Step 1: Configure Terraform and AWS CLI

- Install Terraform on local system.
- Install AWS CLI and configure IAM credentials.
- Verify Terraform and AWS CLI installation.
- Configure AWS region as ap-south-1 (Mumbai).

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Initializing provider plugins found in the state...
- Reusing previous version of hashicorp/aws
- Using previously-installed hashicorp/aws v6.45.0

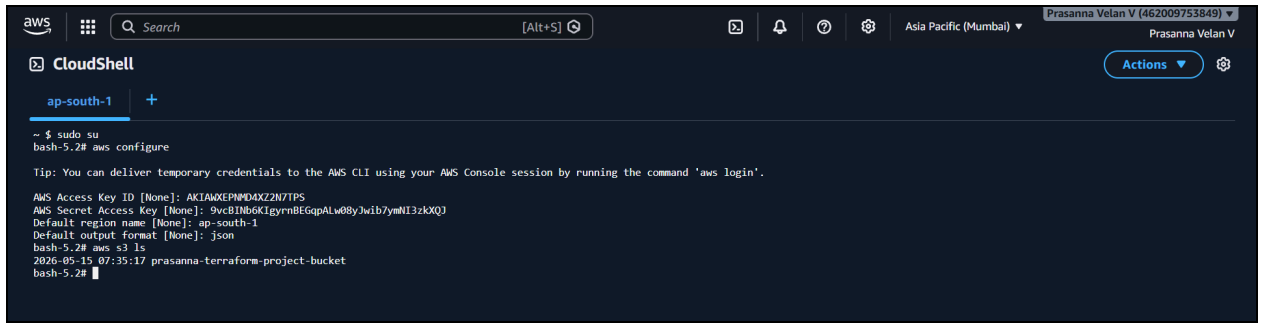
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\prasa\OneDrive\Desktop\aws-3tier-terraform-project>

```

Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture



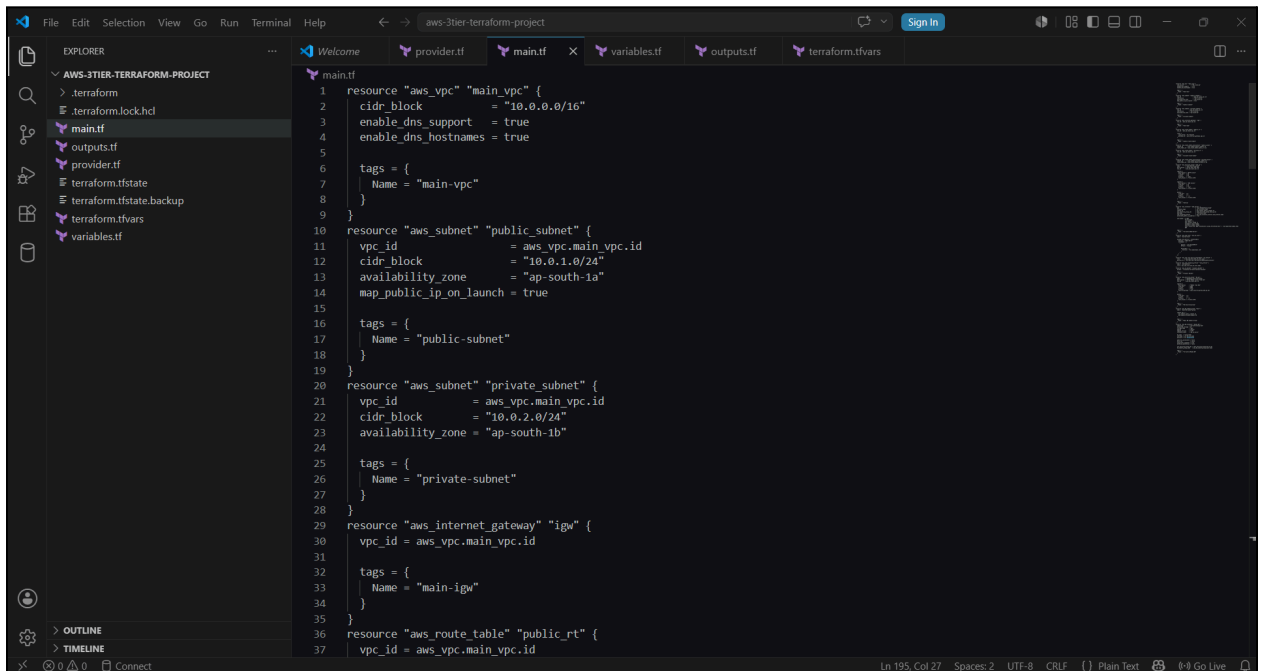
```
aws
[Search] [Alt+S]
CloudShell
ap-south-1 +
~ $ sudo su
bash-5.2# aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: AKIAKXEPNMDXZ2N7TPS
AWS Secret Access Key [None]: 9vcBINb6K1gyrnBEGqpALw08yJwib7ymE3z0KQJ
Default region name [None]: ap-south-1
Default output format [None]: json
bash-5.2# aws s3 ls
2026-05-15 07:35:17 prasanna-terraform-project-bucket
bash-5.2#
```

Step 2: Create Custom VPC Infrastructure

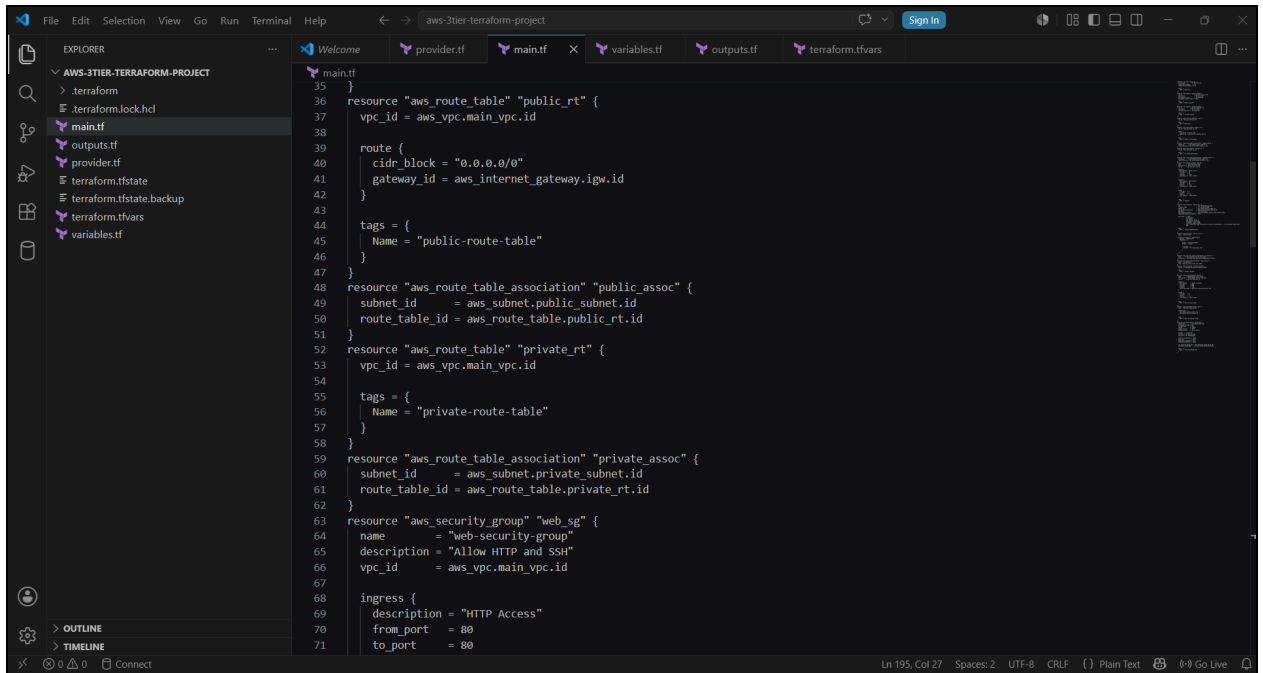
- Create VPC with CIDR block 10.0.0.0/16.
- Create Public Subnet with CIDR 10.0.1.0/24.
- Create Private Subnet with CIDR 10.0.2.0/24.
- Create Internet Gateway and attach it to VPC.
- Configure Route Tables and subnet associations.



```
File Edit Selection View Go Run Terminal Help
aws-3tier-terraform-project
Welcome provider.tf main.tf variables.tf outputs.tf terraform.tfvars
EXPLORER
AWS-3TIER-TERRAFORM-PROJECT
> terraform
> terraform.lock.hcl
main.tf
outputs.tf
provider.tf
terraform.tfstate
terraform.tfstate.backup
terraform.tfvars
variables.tf
OUTLINE
TIMELINE
Ln 195, Col 27 Spaces: 2 UTF-8 CRLF Plain Text 0-9 Go Live
```

```
1 resource "aws_vpc" "main_vpc" {
2   cidr_block      = "10.0.0.0/16"
3   enable_dns_support = true
4   enable_dns_hostnames = true
5
6   tags = {
7     Name = "main-vpc"
8   }
9 }
10 resource "aws_subnet" "public_subnet" {
11   vpc_id            = aws_vpc.main_vpc.id
12   cidr_block        = "10.0.1.0/24"
13   availability_zone  = "ap-south-1a"
14   map_public_ip_on_launch = true
15
16   tags = {
17     Name = "public-subnet"
18   }
19 }
20 resource "aws_subnet" "private_subnet" {
21   vpc_id            = aws_vpc.main_vpc.id
22   cidr_block        = "10.0.2.0/24"
23   availability_zone  = "ap-south-1b"
24
25   tags = {
26     Name = "private-subnet"
27   }
28 }
29 resource "aws_internet_gateway" "igw" {
30   vpc_id = aws_vpc.main_vpc.id
31
32   tags = {
33     Name = "main-igw"
34   }
35 }
36 resource "aws_route_table" "public_rt" {
37   vpc_id = aws_vpc.main_vpc.id
```

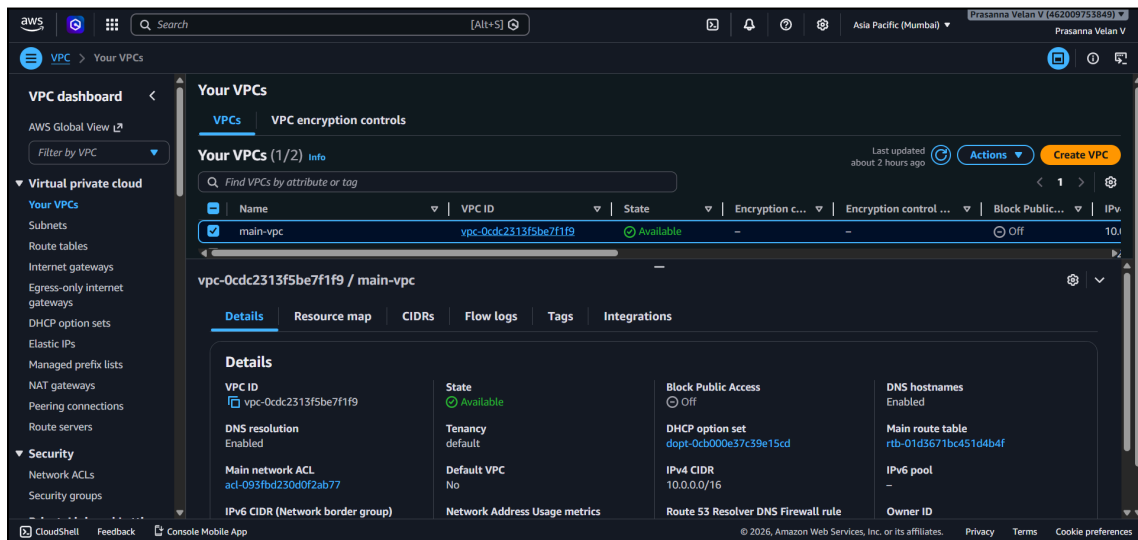
Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture



The screenshot shows a VS Code editor with a project named 'aws-3tier-terraform-project'. The Explorer pane on the left shows the project structure with files like .terraform, .terraform.lock.hcl, main.tf, outputs.tf, provider.tf, terraform.tfstate, terraform.tfstate.backup, terraform.tfvars, and variables.tf. The main editor displays the content of 'main.tf', which defines several AWS resources:

```
35
36 resource "aws_route_table" "public_rt" {
37   vpc_id = aws_vpc.main_vpc.id
38
39   route {
40     cidr_block = "0.0.0.0/0"
41     gateway_id = aws_internet_gateway.igw.id
42   }
43
44   tags = {
45     Name = "public-route-table"
46   }
47 }
48 resource "aws_route_table_association" "public_assoc" {
49   subnet_id = aws_subnet.public_subnet.id
50   route_table_id = aws_route_table.public_rt.id
51 }
52 resource "aws_route_table" "private_rt" {
53   vpc_id = aws_vpc.main_vpc.id
54
55   tags = {
56     Name = "private-route-table"
57   }
58 }
59 resource "aws_route_table_association" "private_assoc" {
60   subnet_id = aws_subnet.private_subnet.id
61   route_table_id = aws_route_table.private_rt.id
62 }
63 resource "aws_security_group" "web_sg" {
64   name = "web-security-group"
65   description = "Allow HTTP and SSH"
66   vpc_id = aws_vpc.main_vpc.id
67
68   ingress {
69     description = "HTTP Access"
70     from_port = 80
71     to_port = 80
```

```
PS C:\Users\prasa\OneDrive\Desktop\aws-3tier-terraform-project> terraform apply
aws_vpc.main_vpc: Refreshing state... [id=vpc-0cdc2313f5be7f1f9]
aws_s3_bucket.project_bucket: Refreshing state... [id=prasanna-terraform-project-bucket]
aws_iam_role.ec2_s3_role: Refreshing state... [id=ec2-s3-role]
aws_iam_role_policy_attachment.s3_attach: Refreshing state... [id=ec2-s3-role/arn:aws:iam::aws:policy/AmazonS3FullAccess]
aws_iam_instance_profile.ec2_profile: Refreshing state... [id=ec2-profile]
aws_subnet.private_subnet: Refreshing state... [id=subnet-0cbc403b902b7a1a8]
aws_route_table.private_rt: Refreshing state... [id=rtb-0acb9d4411e422c59]
aws_internet_gateway.igw: Refreshing state... [id=igw-06c31a29b3ac01ccb]
aws_subnet.public_subnet: Refreshing state... [id=subnet-0ea32500ce4ec6b62]
aws_security_group.web_sg: Refreshing state... [id=sg-0e222fd88134a9dde]
aws_route_table.public_rt: Refreshing state... [id=rtb-0201a0de101b8d086]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-02915889662b125f6]
aws_route_table_association.private_assoc: Refreshing state... [id=rtbassoc-08f18897f9c4a06a3]
```



The screenshot shows the AWS Management Console for the 'Prasanna Velan V (462009753549)' account in the 'Asia Pacific (Mumbai)' region. The 'VPC dashboard' is active, showing a list of VPCs. The 'main-vpc' (vpc-0cdc2313f5be7f1f9) is selected, and its details are displayed.

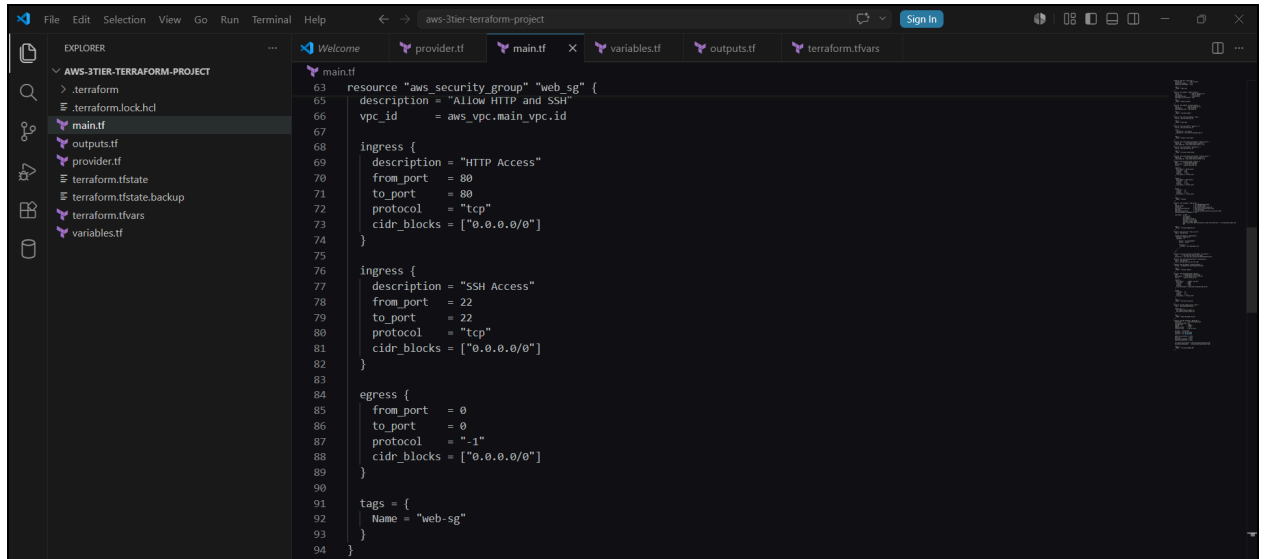
Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv...
main-vpc	vpc-0cdc2313f5be7f1f9	Available	-	-	Off	10.0.0.0/16

vpc-0cdc2313f5be7f1f9 / main-vpc

Details	Resource map	CIDRs	Flow logs	Tags	Integrations
VPC ID vpc-0cdc2313f5be7f1f9 DNS resolution Enabled Main network ACL acl-093fbd230d0f2ab77 IPv6 CIDR (Network border group)	State Available Tenancy default Default VPC No Network Address Usage metrics	Block Public Access Off DHCP option set dopt-0cb000e37c39e15cd IPv4 CIDR 10.0.0.0/16 Route S3 Resolver DNS Firewall rule	DNS hostnames Enabled Main route table rtb-01d3671bc451d4b4f IPv6 pool - Owner ID		

Step 3: Configure Security Groups

- Create EC2 Security Group allowing HTTP (80) and SSH (22).
- Create RDS Security Group allowing MySQL (3306) access only from EC2 Security Group.
- Configure outbound rules for internet access.



```

63 resource "aws_security_group" "web_sg" {
64   description = "Allow HTTP and SSH"
65   vpc_id      = aws_vpc.main_vpc.id
66
67   ingress {
68     description = "HTTP Access"
69     from_port   = 80
70     to_port     = 80
71     protocol    = "tcp"
72     cidr_blocks = ["0.0.0.0/0"]
73   }
74
75   ingress {
76     description = "SSH Access"
77     from_port   = 22
78     to_port     = 22
79     protocol    = "tcp"
80     cidr_blocks = ["0.0.0.0/0"]
81   }
82
83   egress {
84     from_port   = 0
85     to_port     = 0
86     protocol    = "-1"
87     cidr_blocks = ["0.0.0.0/0"]
88   }
89
90   tags = {
91     Name = "web-sg"
92   }
93 }
94

```

```

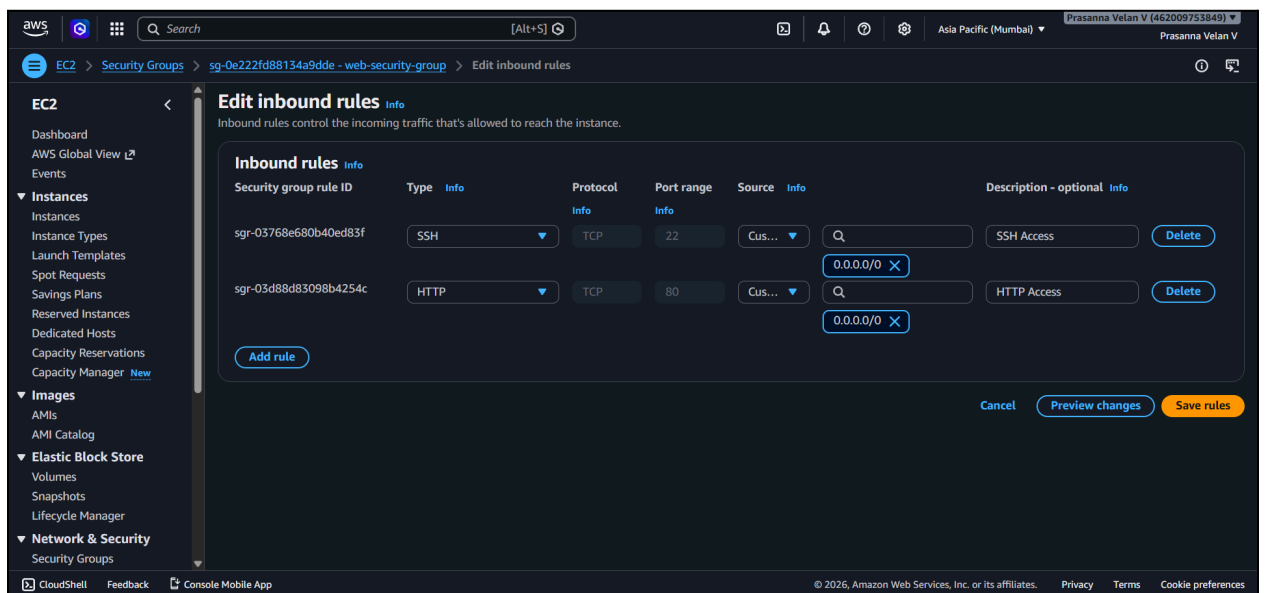
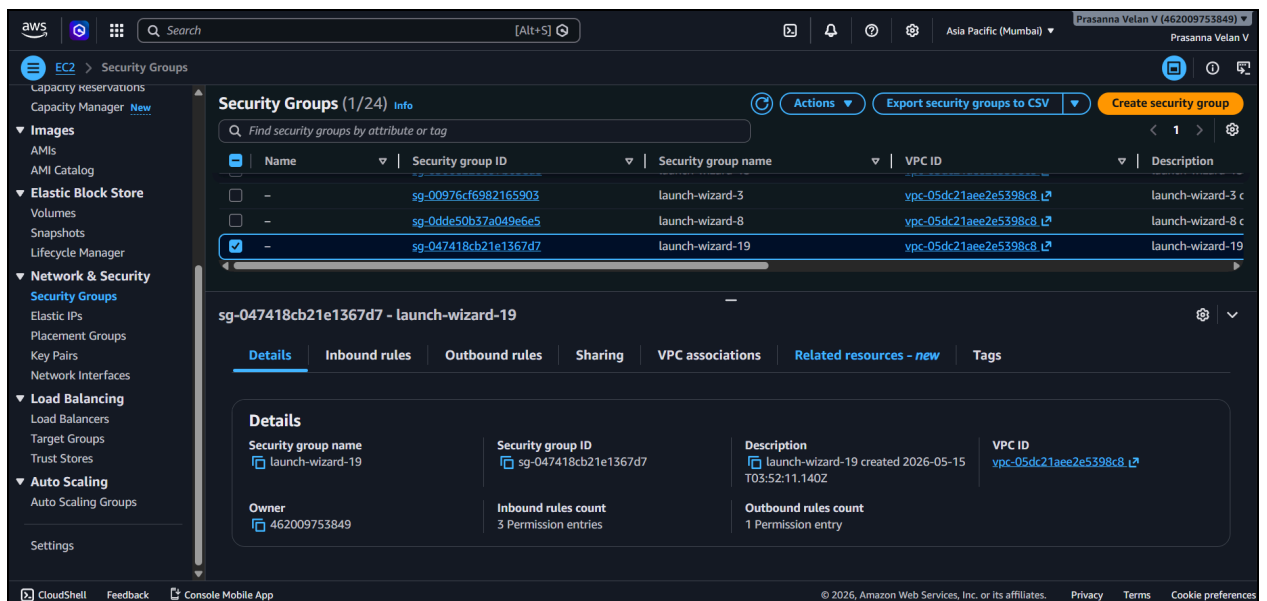
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 16s [id=i-035693cc153911594]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\prasa\OneDrive\Desktop\aws-3tier-terraform-project> terraform fmt
main.tf
terraform.tfvars
PS C:\Users\prasa\OneDrive\Desktop\aws-3tier-terraform-project> terraform validate
Success! The configuration is valid.

PS C:\Users\prasa\OneDrive\Desktop\aws-3tier-terraform-project> terraform plan
aws_vpc.main_vpc: Refreshing state... [id=vpc-0cdc2313f5be7f1f9]
aws_s3_bucket.project_bucket: Refreshing state... [id=prasanna-terraform-project-bucket]
aws_iam_role.ec2_s3_role: Refreshing state... [id=ec2-s3-role]
aws_iam_instance_profile.ec2_profile: Refreshing state... [id=ec2-profile]
aws_iam_role_policy_attachment.s3_attach: Refreshing state... [id=ec2-s3-role/arn:aws:iam::aws:policy/AmazonS3FullAccess]
aws_internet_gateway.igw: Refreshing state... [id=igw-06c31a29b3ac01ccb]
aws_subnet.public_subnet: Refreshing state... [id=subnet-0ea3250ce4ec6b62]
aws_route_table.private_rt: Refreshing state... [id=rtb-0acb9d4411e422c59]
aws_subnet.private_subnet: Refreshing state... [id=subnet-0cbc403b902b7a1a8]
aws_security_group.web_sg: Refreshing state... [id=sg-0e222fd88134a9dde]
aws_route_table.public_rt: Refreshing state... [id=rtb-0201a0de101b8d086]
aws_route_table_association.private_assoc: Refreshing state... [id=rtbassoc-08f18897f9c4a06a3]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-02915889662b125f6]
aws_instance.web_server: Refreshing state... [id=i-035693cc153911594]

```

Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture



Step 4: Launch EC2 Web Server

- Launch Ubuntu EC2 instance in Public Subnet.
- Associate public IP address.
- Install Apache Web Server using Terraform user_data.
- Deploy custom HTML application.
- Verify website accessibility using EC2 public IP.

Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture

```
95 resource "aws_instance" "web_server" {
96     ami             = "ami-0f58b397bc5c1f2e8"
97     instance_type   = var.instance_type
98     subnet_id       = aws_subnet.public_subnet.id
99     vpc_security_group_ids = [aws_security_group.web_sg.id]
100     key_name        = "terraform-key"
101     iam_instance_profile = aws_iam_instance_profile.ec2_profile.name
102     associate_public_ip_address = true
103
104     user_data = <<-EOF
105     #!/bin/bash
106     yum update -y
107     yum install httpd -y
108     systemctl start httpd
109     systemctl enable httpd
110     echo "<h1>3-Tier AWS Architecture using Terraform</h1>" > /var/www/html/index.html
111     EOF
112
113     tags = {
114         Name = "Terraform-Web-Server"
115     }
116 }
```

Plan: 3 to add, 1 to change, 0 to destroy.

Changes to Outputs:

```
+ ec2_public_ip      = "13.233.198.50"
+ private_subnet_id = "subnet-0cbc403b902b7a1a8"
+ public_subnet_id  = "subnet-0ea32500ce4ec6b62"
+ rds_endpoint      = (known after apply)
+ s3_bucket_name    = "prasanna-terraform-project-bucket"
+ vpc_id            = "vpc-0cdc2313f5be7f1f9"
```

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run `"terraform apply"` now.

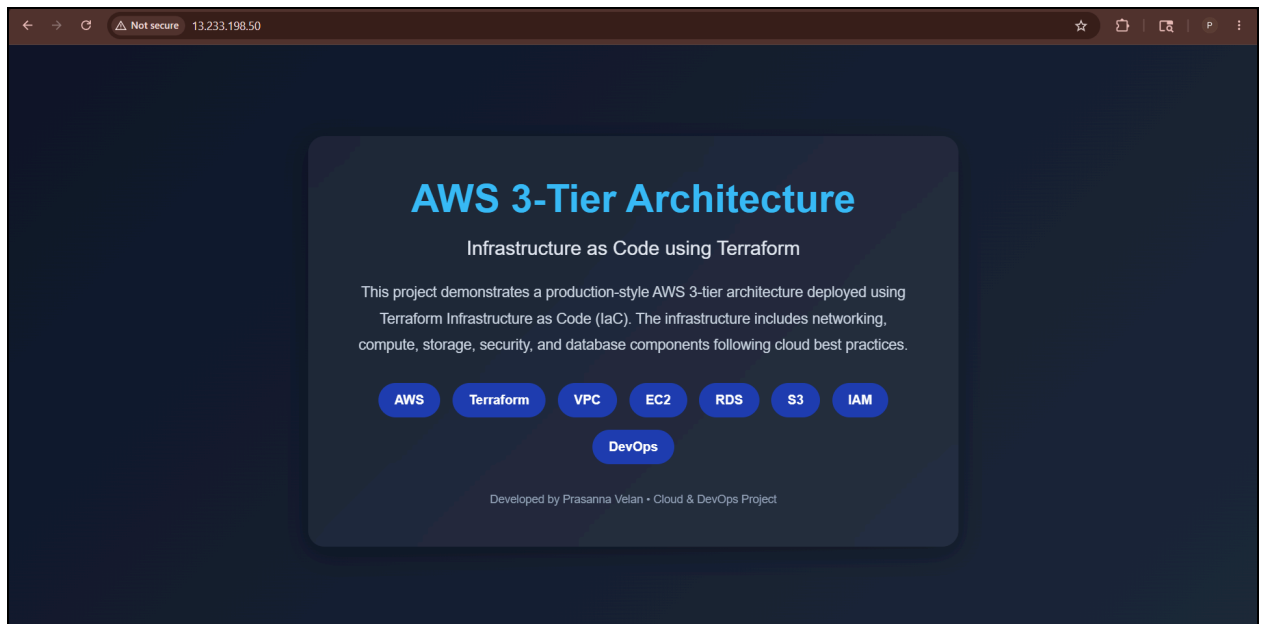
```
PS C:\Users\prasa\OneDrive\Desktop\aws-3tier-terraform-project> terraform apply
aws_iam_role.ec2_s3_role: Refreshing state... [id=ec2-s3-role]
aws_vpc.main_vpc: Refreshing state... [id=vpc-0cdc2313f5be7f1f9]
aws_s3_bucket.project_bucket: Refreshing state... [id=prasanna-terraform-project-bucket]
aws_iam_role_policy_attachment.s3_attach: Refreshing state... [id=ec2-s3-role/arn:aws:iam::aws:policy/AmazonS3FullAccess]
aws_iam_instance_profile.ec2_profile: Refreshing state... [id=ec2-profile]
aws_internet_gateway.igw: Refreshing state... [id=igw-06c31a29b3ac01ccb]
aws_route_table.private_rt: Refreshing state... [id=rtb-0acb9d4411e422c59]
aws_subnet.public_subnet: Refreshing state... [id=subnet-0ea32500ce4ec6b62]
aws_subnet.private_subnet: Refreshing state... [id=subnet-0cbc403b902b7a1a8]
aws_security_group.web_sg: Refreshing state... [id=sg-0e222fd88134a9dde]
aws_route_table.public_rt: Refreshing state... [id=rtb-0201a0de101b8d086]
aws_route_table_association.private_assoc: Refreshing state... [id=rtbassoc-08f18897f9c4a06a3]
aws_route_table_association.public_assoc: Refreshing state... [id=rtbassoc-02915889662b125f6]
aws_instance.web_server: Refreshing state... [id=i-035693cc153911594]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

```
+ create
~ update in-place
```

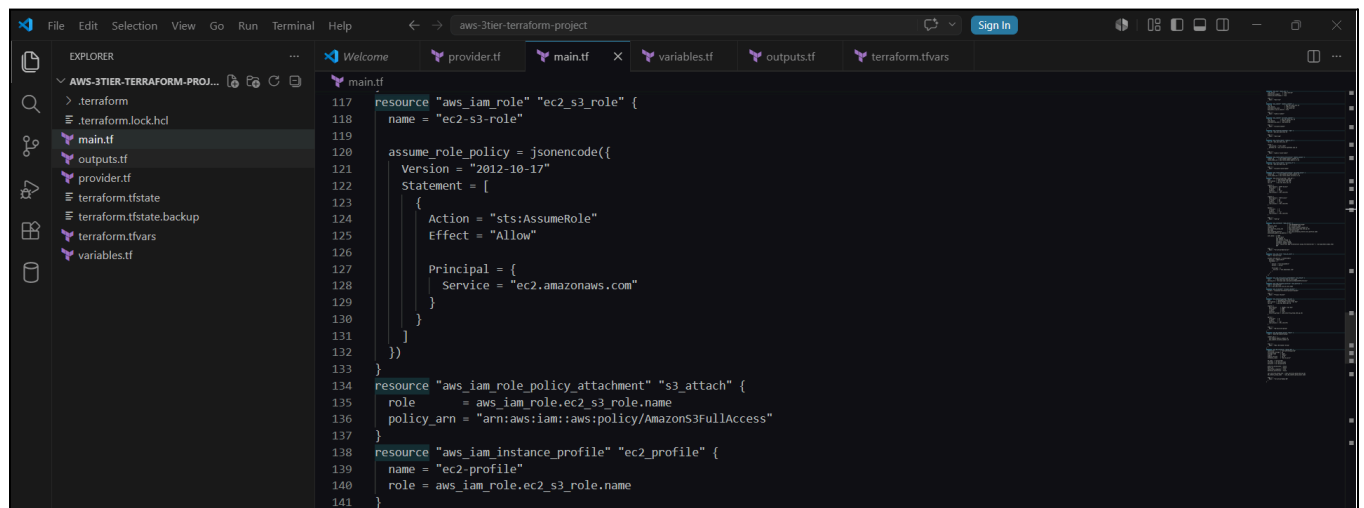
The screenshot shows the AWS Management Console interface for the EC2 service. The 'Instances' page is active, displaying a list of EC2 instances. One instance, 'Terraform-Web-Server' (ID: i-035693cc153911594), is shown in a 'Running' state. The console provides detailed information about this instance, including its public IP address (13.233.198.50), private IP address (10.0.1.12), and hostname (ip-10-0-1-12.ap-south-1.compute.internal). The interface also includes navigation links for various AWS services and a search bar at the top.

Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture

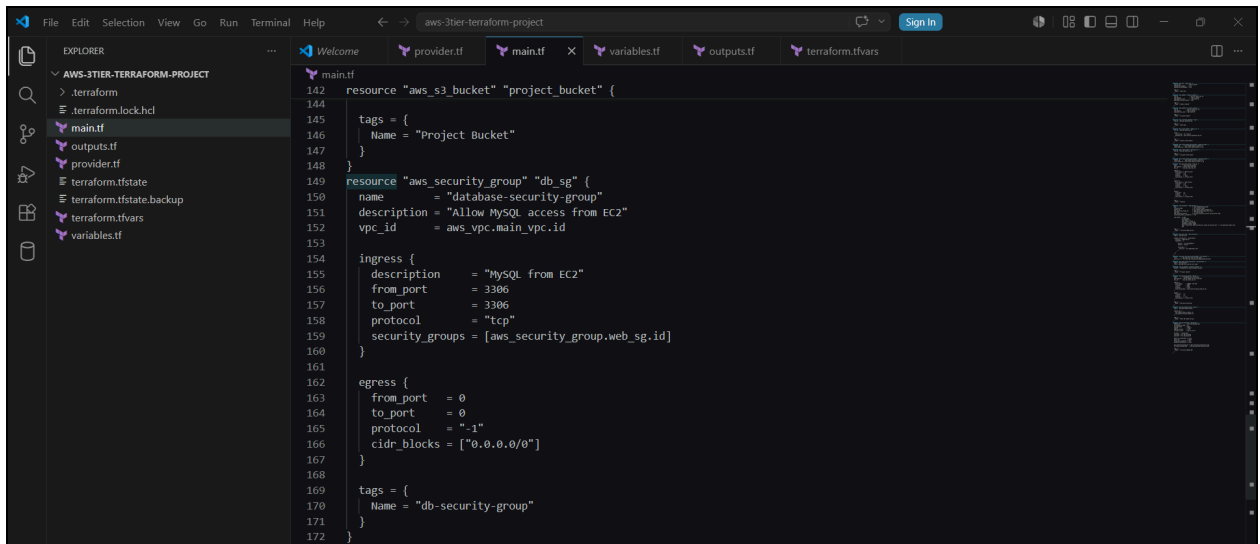


Step 5: Create IAM Role and S3 Bucket

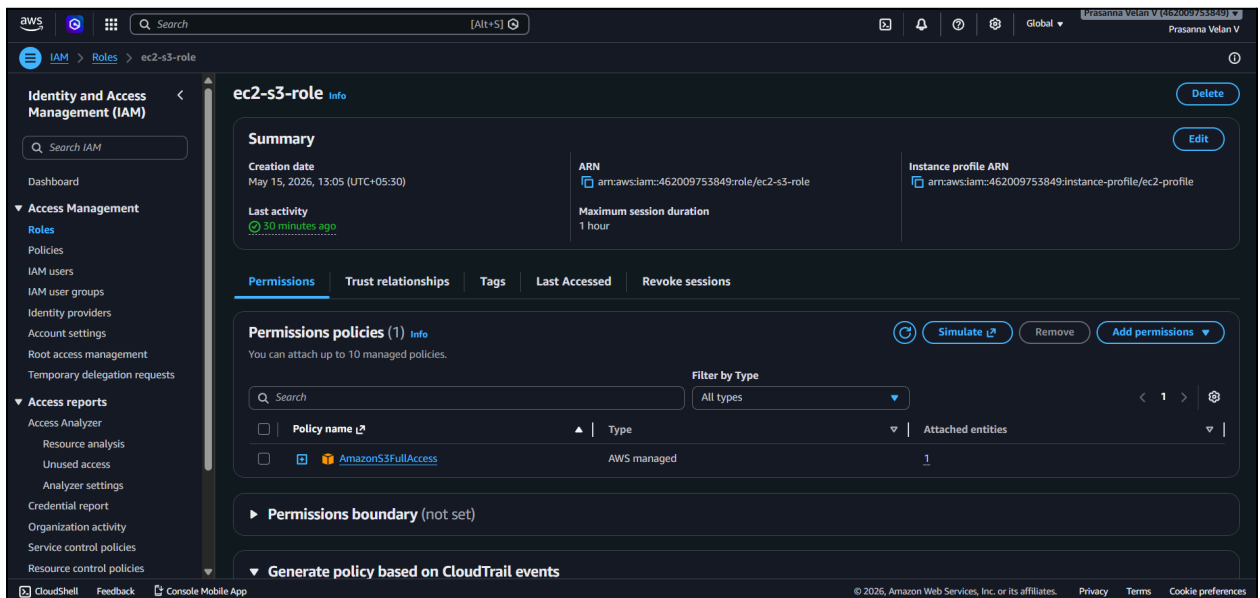
- Create IAM role for EC2 instance.
- Attach AmazonS3FullAccess policy.
- Create IAM Instance Profile and attach it to EC2.
- Create Amazon S3 bucket for storage integration.



Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture



```
main.tf
142 resource "aws_s3_bucket" "project_bucket" {
143   tags = {
144     Name = "Project Bucket"
145   }
146 }
147
148 resource "aws_security_group" "db_sg" {
149   name        = "database-security-group"
150   description = "Allow MySQL access from EC2"
151   vpc_id      = aws_vpc.main_vpc.id
152
153   ingress {
154     description = "MySQL from EC2"
155     from_port   = 3306
156     to_port     = 3306
157     protocol    = "tcp"
158     security_groups = [aws_security_group.web_sg.id]
159   }
160
161   egress {
162     from_port   = 0
163     to_port     = 0
164     protocol    = "-1"
165     cidr_blocks = ["0.0.0.0/0"]
166   }
167
168   tags = {
169     Name = "db-security-group"
170   }
171 }
172 }
```



ec2-s3-role Info

Summary

Creation date May 15, 2026, 13:05 (UTC+05:30)	ARN arn:aws:iam:462009753849:role/ec2-s3-role	Instance profile ARN arn:aws:iam:462009753849:instance-profile/ec2-profile
Last activity 30 minutes ago	Maximum session duration 1 hour	

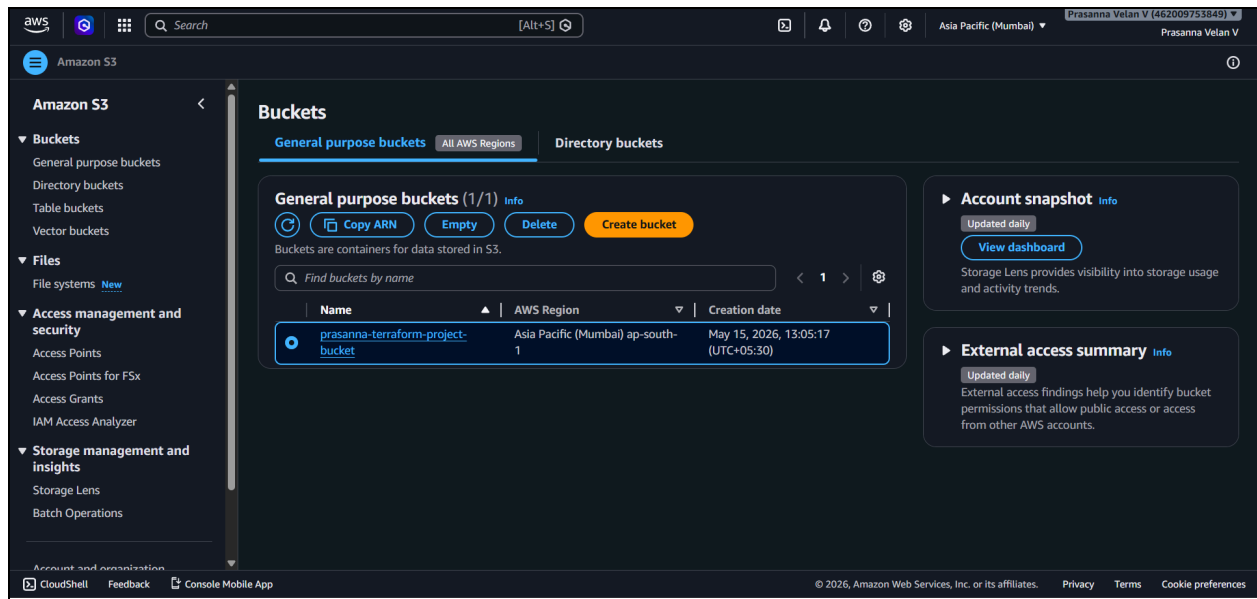
Permissions policies (1) Info

You can attach up to 10 managed policies.

Policy name	Type	Attached entities
AmazonS3FullAccess	AWS managed	1

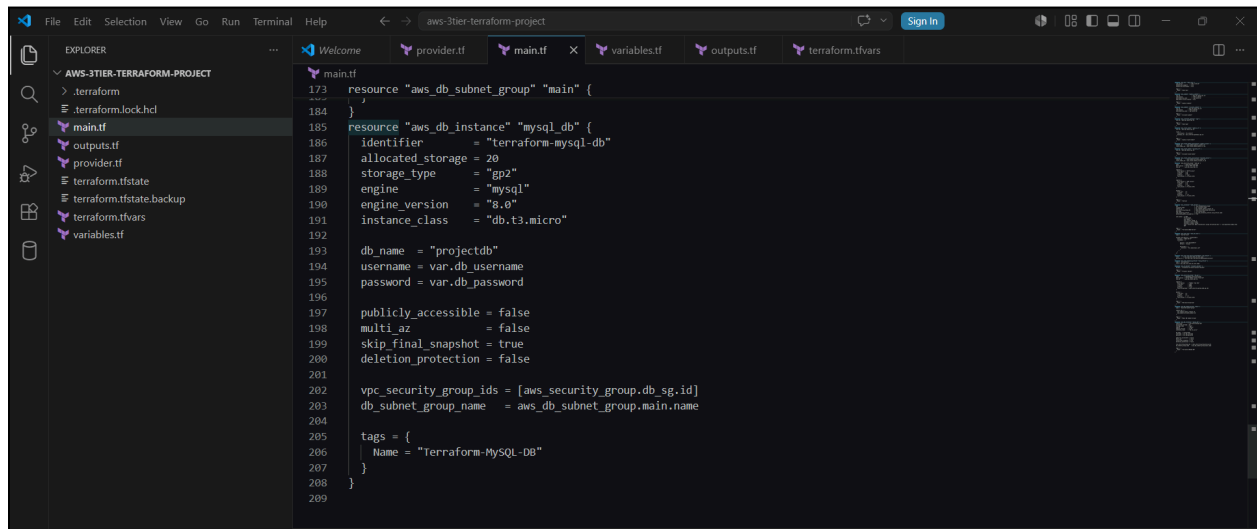
Permissions boundary (not set)

Generate policy based on CloudTrail events

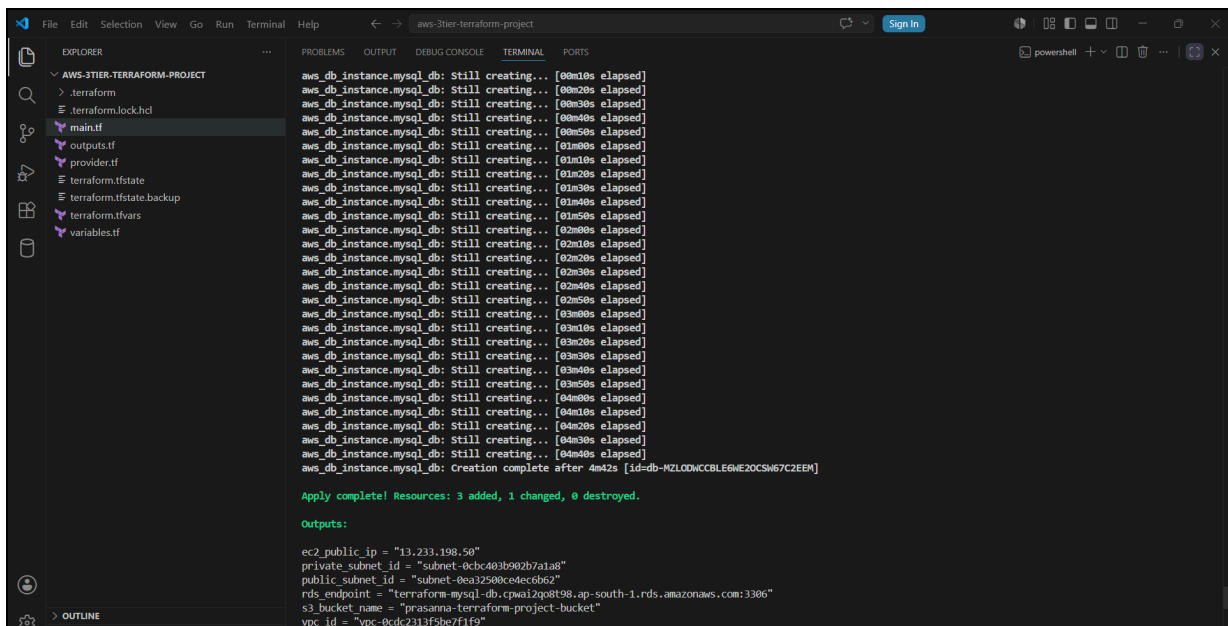


Step 6: Create RDS MySQL Database

- Created DB Subnet Group using public and private subnets.
- Launch Amazon RDS MySQL instance.
- Configure database credentials and security groups.
- Disable public accessibility for enhanced security.



Infrastructure as Code (IaC) using Terraform for AWS 3-Tier Architecture

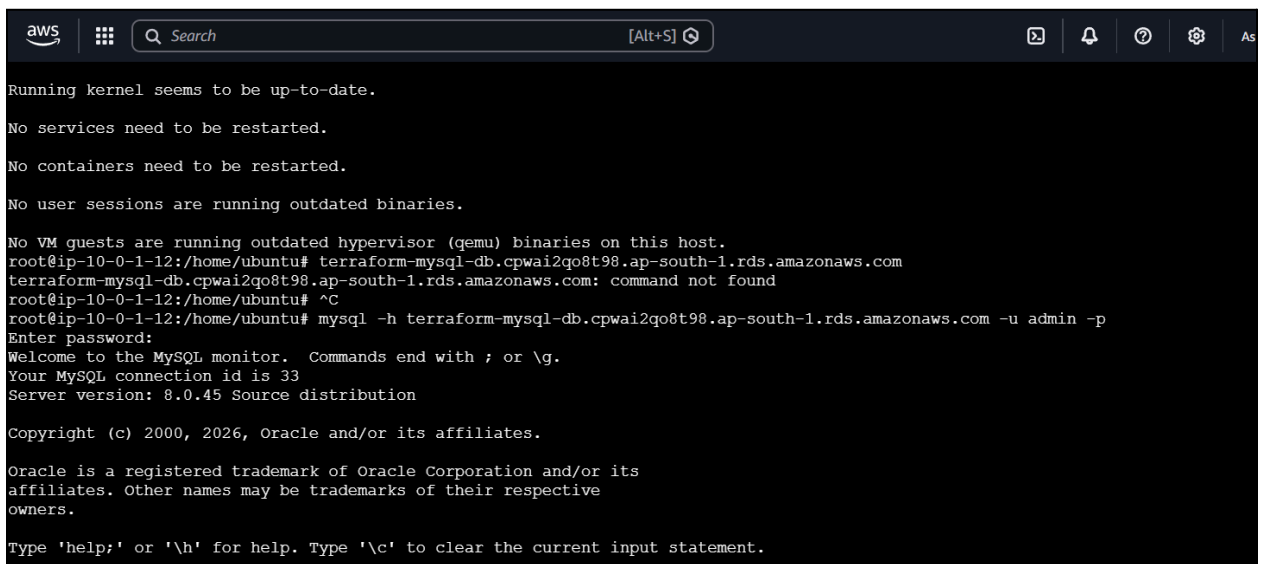


```
aws_db_instance.mysql_db: Still creating... [00m10s elapsed]
aws_db_instance.mysql_db: Still creating... [00m20s elapsed]
aws_db_instance.mysql_db: Still creating... [00m30s elapsed]
aws_db_instance.mysql_db: Still creating... [00m40s elapsed]
aws_db_instance.mysql_db: Still creating... [00m50s elapsed]
aws_db_instance.mysql_db: Still creating... [01m00s elapsed]
aws_db_instance.mysql_db: Still creating... [01m10s elapsed]
aws_db_instance.mysql_db: Still creating... [01m20s elapsed]
aws_db_instance.mysql_db: Still creating... [01m30s elapsed]
aws_db_instance.mysql_db: Still creating... [01m40s elapsed]
aws_db_instance.mysql_db: Still creating... [01m50s elapsed]
aws_db_instance.mysql_db: Still creating... [02m00s elapsed]
aws_db_instance.mysql_db: Still creating... [02m10s elapsed]
aws_db_instance.mysql_db: Still creating... [02m20s elapsed]
aws_db_instance.mysql_db: Still creating... [02m30s elapsed]
aws_db_instance.mysql_db: Still creating... [02m40s elapsed]
aws_db_instance.mysql_db: Still creating... [02m50s elapsed]
aws_db_instance.mysql_db: Still creating... [03m00s elapsed]
aws_db_instance.mysql_db: Still creating... [03m10s elapsed]
aws_db_instance.mysql_db: Still creating... [03m20s elapsed]
aws_db_instance.mysql_db: Still creating... [03m30s elapsed]
aws_db_instance.mysql_db: Still creating... [03m40s elapsed]
aws_db_instance.mysql_db: Still creating... [03m50s elapsed]
aws_db_instance.mysql_db: Still creating... [04m00s elapsed]
aws_db_instance.mysql_db: Still creating... [04m10s elapsed]
aws_db_instance.mysql_db: Still creating... [04m20s elapsed]
aws_db_instance.mysql_db: Still creating... [04m30s elapsed]
aws_db_instance.mysql_db: Still creating... [04m40s elapsed]
aws_db_instance.mysql_db: Creation complete after 4m42s [id=db-MZLODMCCBLE6ME20CSN67C2EEM]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:

ec2_public_ip = "13.233.198.50"
private_subnet_id = "subnet-0cbc403b902b7a1a8"
public_subnet_id = "subnet-0ea32500c0ec0c6b2"
rds_endpoint = "terraform-mysql-db.cpwai2qo8t98.ap-south-1.rds.amazonaws.com:3306"
s3_bucket_name = "prasanna-terraform-project-bucket"
vpc_id = "vpc-0cde2313f5be7f1f9"
```



```
Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-10-0-1-12:/home/ubuntu# terraform-mysql-db.cpwai2qo8t98.ap-south-1.rds.amazonaws.com
terraform-mysql-db.cpwai2qo8t98.ap-south-1.rds.amazonaws.com: command not found
root@ip-10-0-1-12:/home/ubuntu# ^C
root@ip-10-0-1-12:/home/ubuntu# mysql -h terraform-mysql-db.cpwai2qo8t98.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 33
Server version: 8.0.45 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Step 7: Verify Database Connectivity

- Connected to EC2 instance.
- Installed MySQL client.
- Connect EC2 instance to RDS MySQL database.
- Created sample database table and insert records.
- Verified successful communication between EC2 and RDS.

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| projectdb |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> USE projectdb;
Database changed
mysql> CREATE TABLE students (
  ->   id INT PRIMARY KEY AUTO_INCREMENT,
  ->   name VARCHAR(100),
  ->   course VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> INSERT INTO students (name, course)
  -> VALUES ('Prasanna', 'AWS DevOps');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM students;
+----+-----+-----+
| id | name  | course |
+----+-----+-----+
| 1  | Prasanna | AWS DevOps |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

i-035693cc153911594 (Terraform-Web-Server)

PublicIPs: 13.233.198.50 PrivateIPs: 10.0.1.12

Troubleshooting:

Issue 1: Website Not Accessible Through Browser

The EC2 web server was inaccessible due to Apache service installation mismatch between Ubuntu and Amazon Linux.

Solution:

- Verified operating system type.
- Installed Apache2 package using apt package manager.
- Started and enabled Apache2 service.
- Verified security group inbound rules for HTTP access.

Issue 2: MySQL Connection Failed

The EC2 instance could not initially connect to the RDS database.

Solution:

- Verified DB Security Group configuration.
- Allowed MySQL port 3306 from EC2 Security Group.
- Installed MySQL client on EC2 instance.
- Successfully connected using RDS endpoint.

Issue 3: Terraform AMI Compatibility

Hardcoded AMI ID launched Ubuntu instead of Amazon Linux.

Solution:

- Verified launched operating system.
- Updated server configuration commands accordingly.
- Used Ubuntu-compatible Apache installation commands.

Conclusion:

The Infrastructure as Code (IaC) AWS 3-Tier Architecture project successfully automated the deployment of a secure and scalable cloud infrastructure using Terraform. The project implemented AWS networking, compute, database, storage, and security services following industry-standard cloud architecture principles.

The architecture includes a custom VPC, public and private subnets, EC2 web server, RDS MySQL database, S3 storage integration, IAM roles, route tables, and security groups. Terraform simplified infrastructure provisioning, improved consistency, and reduced manual operational effort.

The project demonstrates practical knowledge of AWS cloud services, Infrastructure as Code concepts, networking, security, and automation, making it suitable for DevOps, Cloud Engineer, and AWS-related deployment scenarios.